

EXPERIMENT – 05

Aim: Design and train an autoencoder model for Image Compression

Theory:

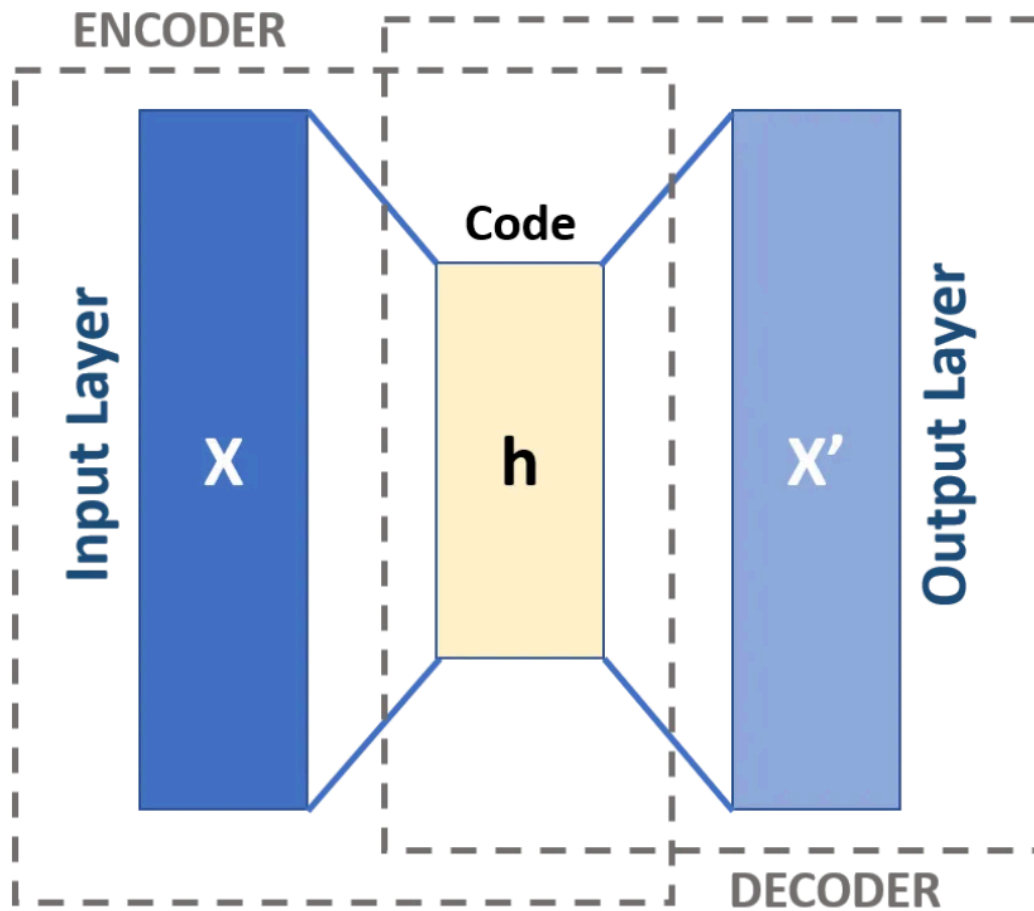
Autoencoders

Autoencoders are a deep learning model for transforming data from a high-dimensional space to a lower-dimensional space. They work by encoding the data, whatever its size, to a 1-D vector. This vector can then be decoded to reconstruct the original data (in this case, an image). The more accurate the autoencoder, the closer the generated data is to the original.

Autoencoders are a deep neural network model that can take in data, propagate it through a number of layers to condense and understand its structure, and finally generate that data again. In this tutorial we'll consider how this works for image data in particular. To accomplish this task an autoencoder uses two different types of networks. The first is called an encoder, and the other is the decoder. The decoder is just a reflection of the layers inside the encoder. Let's clarify how this works.

The job of the encoder is to accept the original data (e.g. an image) that could have two or more dimensions and generate a single 1-D vector that represents the entire image. The number of elements in the 1-D vector varies based on the task being solved. It could have 1 or more elements. The fewer elements in the vector, the more complexity in reproducing the original image accurately.

By representing the input image in a vector of relatively few elements, we actually compress the image. For example, the size of each image in the MNIST dataset (which we'll use in this tutorial) is 28x28. That is, each image has 784 elements. If each image is compressed so that it is represented using just two elements, then we spared 782 elements and thus $(782/784)*100=99.745\%$ of the data.



Working

Fundamentally, AutoEncoders are neural networks. Think of them as consisting of interconnected nodes arranged in layers. Here's a basic breakdown of how they work:

- Input: The input data (say, an image) is fed into the input layer of the autoencoder.
- Encoder: The data travels through the encoder layers. Each layer applies transformations to the data, compressing the information into the hidden code.
- Code: This 'code' lives in the middle layer of the autoencoder, representing a distilled essence of your input data.
- Decoder: The decoding layers take the code and attempt to expand it back into the original representation.
- Output: The output layer produces the autoencoder's reconstruction of the input data.

- **Learning:** The loss (difference) between the input and the reconstruction is calculated. During training, the autoencoder's weights are continuously updated to minimize this loss.

Hyperparameters

Let's dive into the the most important hyperparameters for training AutoEncoders:

- **Bottleneck Size:** The number of neurons in the bottleneck layer (the smallest, middle layer) is crucial. It determines how much the input data is compressed. Fewer neurons mean more compression.
- **Number of Layers and Layer Sizes:** This refers to how many layers are in the AutoEncoder and how many neurons are in each layer. The structure typically narrows down to a smaller 'bottleneck' layer in the middle, which is where the data gets compressed.
- **Activation Functions:** These are functions like ReLU or sigmoid used in the neurons. They help the autoencoder learn non-linear patterns in the data.
- **Loss Function:** This is a way to measure how well the autoencoder is rebuilding the input data. Common choices are Mean Squared Error for continuous data or binary cross-entropy for binary data.
- **Regularization Techniques:** These are methods like L1/L2 regularization or dropout. They are used to prevent the autoencoder from memorizing the input data too closely (overfitting), helping it to generalize better.

The example demonstrates the core function of AutoEncoders: learning a compressed and informative representation of data. In this specific case, we're working with the MNIST dataset, which consists of images of handwritten digits.

Dimensionality Reduction: The original images have 784 pixels (28 pixels x 28 pixels). The autoencoder's goal is to identify the most essential features of these handwritten digits and encode them in a much smaller code (e.g., 64 dimensions in our example).

Reconstruction: With just this compressed code, the autoencoder aims to reconstruct the original images as accurately as possible. The success of the reconstruction highlights how well it has managed to capture the essence of the digits.